AI ENGINEERING
AI-Powered products

MÓDULO 2: Arquitecturas CAG

Sesión 5: Funcionalidades avanzadas

Mentor: Julián Fernández
LIDR.co, training the Tech Leaders' of the future

L1DR

Hoy: de un chat que estima a un sistema que se audita

Todo el código ya está escrito: hoy se abre, se explica y se ejecuta

Hora 1 · la base conversacional

Ejercicio · 15 min
Memoria, metadata, endpoints y adjuntos

Bloque 1 · compresión
Resumen + anclas + ventana

Bloque 2 · tier dinámico
La audiencia se deduce del contexto

Bloque 3 · golden dataset
Evaluar como una suite de tests

Hora 2 · Actor-Critic-Boss

Bloque 4 · el patrón
Tres roles y los dos fallos que evita

Bloques 5-6 · Actor y Critic
Mismo actor, crítica estructurada

Bloque 7 · Boss
Aceptar, iterar o sintetizar

Bloque 8 · trade-offs
Cuándo compensa y cuándo sobra

Lo que traéis del ejercicio previo

Cuatro piezas sobre las que se monta toda la sesión

A · Memoria conversacional

`ConversationHistory`: ventana de **6 pares** user/assistant. El system prompt **no** se guarda: se regenera cada turno.

B · Extractor de metadata

Un LLM barato saca `ProjectMetadata` en cada turno. Fusión: lo escalar se pisa, las listas se suman. Si falla, se queda la anterior.

C · Endpoints /sessions

`POST /sessions` · `GET /sessions/{id}` · `POST …/estimate` en **multipart** (texto + ficheros).

D · Adjuntos PDF/DOCX

Extracción local con `pypdf` y `python-docx`, truncado y envuelto entre marcas `--- attachment ---`.

</> En el código app/sessions/**models.py** (l. 27-99) · sessions/**metadata_extractor.py** (l. 26) · routers/**sessions.py** (l. 69-204) · attachments/**extractor.py** (l. 32-123)

El orden de los mensajes no es negociable

Y los cinco errores que más salieron en las correcciones

services/estimation.py · cada turno

```
messages = [{"role": "system",
             "content": system_prompt}]
messages.extend(session.history.to_messages())
messages.append({"role": "user",
                 "content": user_message})
```

Por qué el system va fuera del historial

Se re-renderiza cada turno con la metadata **actual**. Guardarlo congelado sería un bug: el modelo seguiría con el tier y el contexto del turno 1.

Errores frecuentes

- Olvidar `python-multipart`: el endpoint revienta al arrancar.
- Llamar sin crear sesión antes: `404` que parece un bug del servidor.
- Omitir el `system`: se pierden tier, metadata y reglas.
- Adjuntos de 80k caracteres sin truncar: revientan la ventana.
- Extraer metadata antes de comprimir.

</> En el código app/services/**estimation.py** · `estimate_conversational()` (l. 171) · mensajes en l. 221-223 · tests: `tests/test_sessions_window.py`

L1DR

Adjuntos: multimodal directo o extracción local

El proyecto se queda con el camino B

A · PDF directo al modelo

Cómodo, pero **acopla al proveedor**: el multimodal de OpenAI y el de Anthropic no son iguales. Cambias de proveedor y reescribes la integración.

B · Extraer el texto en local

El wrapper del LLM sigue **agnóstico**: solo recibe texto. El extractor es un servicio aparte que mañana puede ser OCR o RAG sin tocar la estimación.

Las marcas dicen al modelo qué dijo el cliente y qué leyó del PDF · una página rota no tumba el PDF

```
--- attachment: brief-cliente.pdf ---
... texto extraído (truncado a MAX_ATTACHMENT_CHARS) ...
--- end attachment ---
```

</> En el código app/attachments/**extractor.py** · `_extract_pdf()` (l. 39), `_extract_docx()` (l. 55), `extract_text()` (l. 63), `enrich_transcript()` (l. 107)

L1DR

Estrategias de historial

Qué hacer cuando la conversación no cabe

Estrategia	Qué conserva	Qué arriesga	Coste
Ventana deslizante	los últimos N turnos	olvida lo que se pactó al principio	el ejercicio
Resumen acumulado	lo antiguo, condensado	pierde la letra exacta de lo crítico	LLM barato
Anclas	turnos críticos, literales	si se ancla de más, el prompt crece	8 reglas, sin LLM
Híbrida	**resumen + anclas + ventana**	una llamada extra al comprimir	**hoy**

El fallo de la ventana

En el turno 7 desaparece el turno 1: «firmamos NDA», «presupuesto tope 50k». El modelo deja de tratarlo como confidencial.

La idea de hoy

Lo crítico se queda **literal** (ancla), lo demás se **resume**, y lo reciente sigue **vivo** en la ventana.

</> En el código

app/sessions/**models.py** · `ConversationHistory` (l. 37): `messages`, `anchors`, `summary` · app/config.py · `MAX_CONVERSATION_TURNS = 6` (l. 43)

Compresión híbrida: cómo se compone el prompt

Turno 8 con max_turns = 6 y un NDA en el turno 1

La ventana solo limita el tramo vivo

```
to_messages() → [
  {user: "[Earlier conversation summary]
          El cliente describió un CRM…"},    ← resumen
  NDA · turno 1 (user + assistant),          ← ancla
  turnos 3 … 8                               ← ventana
]
```

El resumen entra como `user`

Ya hay un `system` y se regenera cada turno: un segundo `system` rompería el contrato. El corchete le dice al modelo que es contexto.

El cambio de contrato: `append()` ya no recorta

Antes, `append` recortaba en silencio. Ahora solo añade, y el servicio llama justo después a `apply_compression(...)`: quien lee el código ve **cuándo** se paga la llamada al LLM. Un solo resumen por pasada, rehecho entero a partir del anterior (nada de resumen de resúmenes).

</> En el código sessions/**models.py** · `to_messages()` (l. 73) · `append()` (l. 59) · compression/**policy.py** · `apply()` (l. 44), `apply_compression()` (l. 90)

Anclas: ocho reglas, cero LLM

Mejor quedarse corto que anclar de más

_HEURISTIC_PATTERNS (resumido)

`nda`	`nda | non-disclosure | under embargo`
`signed_contract`	`signed the contract | sow | msa`
`scope_frozen`	`scope is frozen | locked | final`
`budget_locked`	`budget is capped at …`
`compliance`	`hipaa | gdpr | sox | pci-dss | iso-27001`
`deadline_hard`	`hard deadline | must go live by`
`contractual`	`contractually bound | required`
`explicit_commitment`	`we agreed to | the client committed to`

Por qué tan pocas

Un falso negativo acaba en el resumen. Un falso positivo se queda **literal en el prompt para siempre**: eso es lo caro.

Solo se mira al usuario

El assistant entra «por arrastre» si su turno es ancla. Cada promoción deja en el log la regla que la disparó (`anchor_detected`). `ANCHOR_DETECTION_MODE=llm` para paráfrasis densas.

</> En el código app/sessions/compression/**anchors.py** · `_HEURISTIC_PATTERNS` (l. 35), `detect()` (l. 88), modo LLM opcional `_detect_llm()` (l. 110)

Demo: ocho turnos en la misma sesión

Turno 1 en inglés con «we signed the NDA»; turnos 2-8, refinamientos

Turno	Mensajes	Anclas	Resumen	
1-6	2 → 12	0	0	la ventana se llena
7	12	**2**	0	sale el turno 1: es ancla (`nda`)
8	12	2	**908**	sale el turno 2: va al resumen

Ejecución real

gpt-4o, `max_turns=6`: la ventana nunca pasa de 12 mensajes, el NDA sobrevive literal y el resto queda en 908 caracteres de resumen.

Lo que no hace el ancla

En el turno 1 el tier es **executive** (`nda_detected`); del 2 en adelante vuelve a **default**. El resolver de tier mira el texto del turno y la metadata, **no las anclas**. Buena pregunta para la clase: ¿debería?

</> En el código

Interfaz: panel lateral (Window, Anchors kept, Summary) · `GET /sessions/{id}` · routers/**sessions.py** · `SessionInfoResponse` (l. 57)

Tiers: la misma estimación para audiencias distintas

Un solo prompt, un bloque <audience> que cambia

Tier	Para quién	Cómo cambia la respuesta
executive	sponsor, dirección	cifra defendible, riesgos primero, 3-4 fases, sin jerga
pm	jefe de proyecto	hitos, quién y qué entrega, máx. 4 fases, confianza prudente
developer	tech lead	stack, integraciones, riesgo técnico por fase, hasta 6-8 fases
default	audiencia mixta	equilibrio negocio / técnica, sin ajustes

Estático (el artículo)

El cliente pasa `tier=executive`. Funciona si acierta, pero en una conversación larga la audiencia cambia: empieza el comercial y termina el CTO.

Dinámico (hoy)

Se deduce de pistas: NDA, normativa, densidad técnica, tamaño del equipo. **Jinja en vez de cuatro prompts**: un arreglo común se hace una vez.

</> En el código app/prompts/estimation/v3/**system.j2** · bloque `<audience>` (l. 17-39) · app/sessions/**tier_resolver.py** · `class Tier` (l. 30)

El resolver: reglas en orden, la primera que acierta gana

Y un override humano que siempre manda

tier_resolver.py · el orden importa

```
_RULES = (
  TierRule("nda_detected",        EXECUTIVE),
  TierRule("regulatory_context", EXECUTIVE),
  TierRule("technical_audience", DEVELOPER),
  TierRule("low_budget_pm",       PM),
)  # si ninguna → DEFAULT
```

Devuelve (tier, regla)

La regla viaja al panel: «executive (nda_detected)». Si el tier no es el esperado, el porqué está al lado.

Demo · ejecución real

```
e-commerce · Stripe · 4 personas
  → tier: default    rule: default

"Heads-up, firmamos NDA…" + lo mismo
  → tier: executive  rule: nda_detected

microservicios + Kubernetes + Kafka
  → tier: developer  (≥ 2 palabras técnicas)
```

Lo que vimos al probarlo

«Equipo de 2 personas» no dispara `pm` en el **primer** turno: el tamaño del equipo llega a la metadata **después** de resolver el tier. Desde el segundo turno, sí.

</> En el código app/sessions/**tier_resolver.py** · regex (l. 53-63) · `_RULES` (l. 101) · `resolve_tier()` (l. 109) · llamada en services/estimation.py (l. 199)

Golden dataset: evaluar como una suite de tests

16 casos comiteados en el repo, con rangos y no igualdades

Un caso feliz

```
{
  "id": "case-001-web-saas-crm",
  "transcript": "We want a B2B CRM called Nimbus
     for the sales team. Contacts, deal pipeline,
     basic reporting. Stack: React + Postgres.
     Team of 3.",
  "project_type": "web_saas",
  "expected_in_summary": ["CRM"],
  "expected_technologies_any_of": ["React", "Postgres"],
  "expected_phase_count_range": [3, 6],
  "expected_cost_range_eur": [15000, 80000],
  "expected_duration_weeks_range": [4, 16]
}
```

Rangos, no números exactos

El modelo varía de forma legítima entre ejecuciones. El test caza lo que se sale de proporción, no cada euro.

Tres reglas de oro

- El dataset vive **comiteado**: se puede hacer diff.
- Métricas **deterministas** siempre que se pueda.
- Un juez LLM complementa, nunca es el primer escalón.

</> En el código estimator/evals/**golden_dataset.json** (16 casos) · evals/**dataset.py** · `GoldenCase` (l. 18), `load_dataset()` (l. 46)

L1DR

Tres métricas binarias y lo que dijeron

Ocho casos contra el modo actor, con gpt-4o

Las métricas

- **schema_adherence**: fases suman el total, prefijo «Out of scope» si la confianza baja de 30, nº de fases en rango.
- **cost_bounds**: coste y semanas en rango.
- **content_recall**: aparecen los términos y alguna tecnología.

Caso	Schema	Coste	Recall
001 CRM	PASS	PASS	PASS
002 fidelización	PASS	PASS	PASS
003 RRHH interna	PASS	PASS	PASS
004 ETL datos	PASS	**FAIL**	PASS
005 vago	PASS	PASS	PASS
006 inyección	PASS	PASS	PASS
007 NDA	**FAIL**	**FAIL**	PASS
008 HIPAA	PASS	**FAIL**	PASS

5/8 pasan todo. La lectura útil está en los fallos

El 007 (NDA, tier executive) sale «Out of scope» con confianza baja: el bloque executive pide honestidad y el modelo se retrae. Con gpt-4o-mini, el 004 daba **502**: no conseguía que las fases sumaran el total ni con 6 reintentos. **Sin el dataset no lo habríamos visto.**

</> En el código

evals/**metrics.py** · Schema (l. 26), CostBounds (l. 65), ContentRecall (l. 103) · `docker compose -p aieng-s05live exec estimator python -m evals.run --mode actor --limit 8`

Actor-Critic-Boss: tres roles, separación estricta

Del artículo «Building Effective Agents» (Anthropic)



Fallo 1 · bucle infinito

Actor y crítico solos no tienen condición de parada: el crítico siempre encuentra algo. El Boss pone el **límite** y el plan B.

Fallo 2 · sesgo de confirmación

Con crítica en texto libre el actor se justifica en vez de corregir. Con crítica **estructurada** sabe qué campo concreto rehacer.

</> En el código `POST /sessions/{id}/estimate-acb` · routers/**sessions.py** (l. 203) · services/**estimation.py** · `estimate_with_acb()` (l. 277)

Actor: la pieza que ya teníamos, con otro contrato

No hay código nuevo de actor; cambia qué hace con su salida

1 · Guardrail de entrada	**2** · Resolver tier	**3** · Prompt v3	**4** · [system] + historial + user
5 · LLM estructurado	**6** · Guardrail de salida	**7** · append + comprimir	**8** · Actualizar metadata

El orden 6 → 7 → 8 importa

Si guardas antes del guardrail, una respuesta dudosa entra en la historia como si nada. Si comprimes antes de la metadata, puedes perder hechos aún sin consolidar.

El mismo actor en los dos modos

En `/estimate` su salida es la respuesta. En `/estimate-acb` es un **candidato**. La única diferencia: el template recibe `critic_feedback` y Jinja decide si añade el bloque. **Cero ramas en Python.**

</> En el código services/**estimation.py** · `estimate_conversational()` (l. 171) · closure `_actor()` (l. 320) · prompts/estimation/v3/**user.j2** · `<critic_feedback>` (l. 7)

Critic: feedback estructurado, nunca texto libre

Instructor valida el formato; el prompt lleva el criterio

schemas/critic.py

```
class CriticIssue(BaseModel):
    category: Literal["math_error", "hallucination",
        "scope_mismatch", "phase_imbalance",
        "missing_assumption", "unrealistic_estimate",
        "tier_mismatch"]
    severity: Literal["critical", "major", "minor"]
    field_path: str      # "phases[2].cost_eur"
    description: str
    suggested_fix: str | None
```

`field_path` es la clave

Un crítico útil dice **dónde** está el defecto. El Boss lo pega tal cual en el siguiente prompt del actor: sabe qué campo tocar.

Veredicto: accept · needs_iteration · reject

Validador: iterar exige al menos un problema **critical** o **major**. Por detalles menores no se gasta otra llamada.

Fallback elegante

Si el crítico revienta, devuelve `accept` con `confidence_in_review=0`: el usuario recibe el borrador del actor y el log delata que fue degradado.

</> En el código app/schemas/**critic.py** · categorías (l. 17), `CriticFeedback` (l. 50) · services/**critic.py** · `review()` (l. 33) · prompts/critic/v1/**system.j2**

Boss: tres salidas en nueve líneas

Un autómata sobre el veredicto y el presupuesto de iteraciones

Crítico dice	Rondas	Boss decide	Qué pasa
accept	—	**accept**	devuelve el borrador
reject	—	**synthesize**	no tiene arreglo con este actor
needs_iteration	quedan	**iterate**	vuelve al actor con el feedback
needs_iteration	no quedan	**synthesize**	lo intentó y no convergió

Stateless

El Boss recibe **funciones** (actor y crítico), no objetos: en los tests se inyectan falsos sin mocks complicados. No recuerda nada entre estimaciones.

Synthesize no tira el trabajo

Devuelve el último borrador del actor con los avisos del crítico delante («⚠ Open caveats…») y la confianza a la mitad, con suelo de 30 para no caer en «Out of scope». La honestidad viaja en `final_decision`. En la sesión solo se guarda la respuesta final: los borradores son de usar y tirar.

</> En el código

services/**boss.py** · `run()` (l. 50), `_decide()` (l. 106), `_synthesize_fallback()` (l. 117) · config.py · `BOSS_MAX_ITERATIONS = 3` (l. 66)

Demo ACB: las tres rutas en ejecuciones reales

Crítico con gpt-4o y con gpt-4o-mini

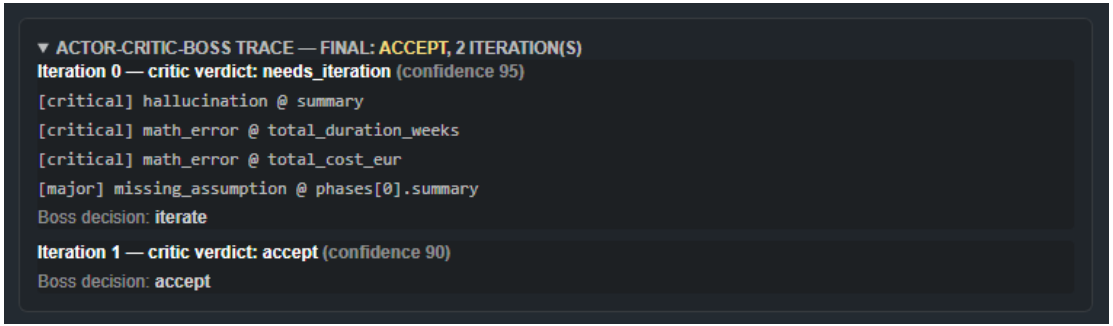
Crítico gpt-4o

```
CRM + facturación con Stripe
  iteración 0 · accept (90)
  → accept · 46.000 €

marketplace: pagos, KYC, chat, app…
2 personas en 6 semanas
  ejecución A · reject → synthesize
  ejecución B · needs_iteration → accept
```

Crítico gpt-4o-mini

```
CRM + facturación con Stripe
  iteración 0 · needs_iteration
  iteración 1 · needs_iteration
  iteración 2 · needs_iteration
  → synthesize (sin rondas) · 57.000 €
  [critical] math_error @ total_cost_eur
  aunque las fases sí sumaban 57.000
```



Dos lecturas

La misma entrada no sale igual dos veces. Y un crítico pequeño da **falsos positivos** (un `math_error` que no existe) que queman rondas.

</> En el código

Interfaz: Demo presets → «ACB · CRM + Stripe» y «ACB · marketplace imposible» · traza desplegable bajo el resultado · `CRITIC_MODEL` en `estimator/.env`

LIDR

Trade-offs: ¿compensa el ACB?

Los mismos ocho casos del golden dataset en los dos modos

Modo	Casos OK	Llamadas LLM / turno
actor	**5/8**	2 (actor + metadata)
acb	**6/8**	3-7 (+ crítico y rondas)

Dónde ganó

El 004 (ETL) se salía del rango de coste con el actor solo y entró con el ACB. El 007 (NDA) falla en los dos: el crítico no arregla un prompt que empuja a «Out of scope».

No usarlo

- Estimaciones rápidas de baja confianza.
- Conversaciones de 1-2 turnos.
- Cuando el coste por llamada pesa más que la calidad marginal.

Sí usarlo

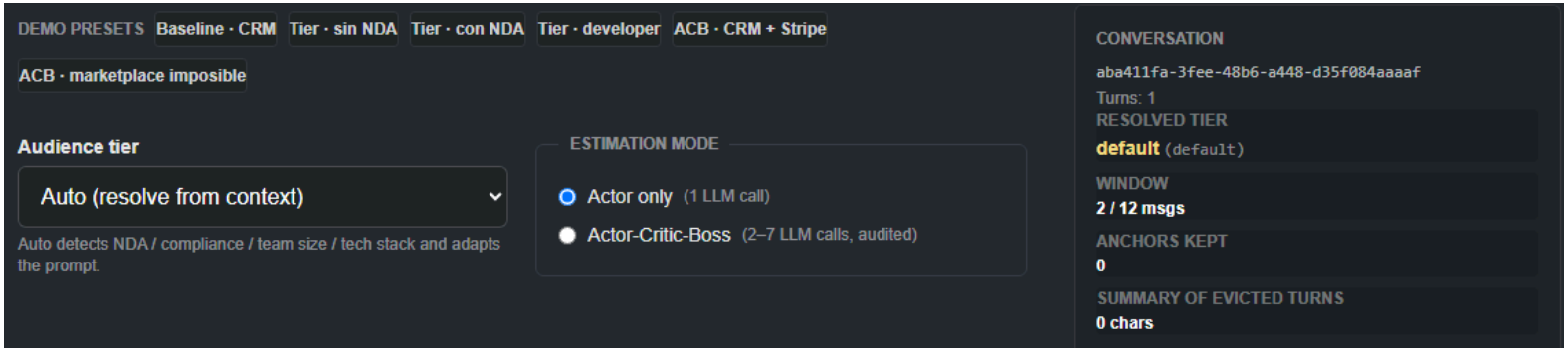
- Cliente importante o contrato grande.
- Proyectos con NDA o normativa: un error de cifra cuesta caro.
- Cualquier agente que escribe y puede contradecirse.

</> En el código

`… exec estimator python -m evals.run --mode actor --limit 8` · `… --mode acb --limit 8` · resultados en `estimator/evals/actor.json` y `acb.json`

La interfaz: todo esto sin tocar Python

Cliente Rails en localhost:3000



Qué hay en pantalla

- **Demo presets**: un clic rellena cada demo de hoy · **Audience tier**: auto o forzado (el override) · **Estimation mode**: actor o Actor-Critic-Boss.
- Panel lateral: tier y regla, ventana, anclas y caracteres de resumen. Bajo el resultado, la traza del ACB.
- Cualquier stack lo replica: multipart a la API, dos endpoints y un `GET /sessions/{id}`. El cliente nunca habla con OpenAI directamente.

</> En el código estimator-web/app/views/chat_sessions/**_form.html.erb** (tier y modo) · **_metadata_panel** · **_estimation_result** (traza) · services/estimator_ai/**client.rb**

Lo que cierras hoy

Hoy

- Memoria que no olvida lo pactado: resumen + anclas + ventana.
- La audiencia se deduce del contexto, con override humano.
- Un golden dataset que caza regresiones antes que el cliente.
- Actor-Critic-Boss: auditar en vez de inventar.

Para pensar

- ¿Qué debería ser ancla en vuestro dominio?
- ¿Cuándo confiar en el resolver y cuándo forzar el tier?
- ¿Qué octava categoría añadiríais al crítico?
- ¿Qué cuarto rol añadiríais al patrón?

Sesión 6 · Data-driven AI

Análisis, formateo y normalización de los datos que alimentan al estimador: el primer paso para que los adjuntos de hoy acaben en un RAG de verdad. El código de referencia de hoy está en la rama `session_05_live`.

✋Tiempo para el feedback con ROTI

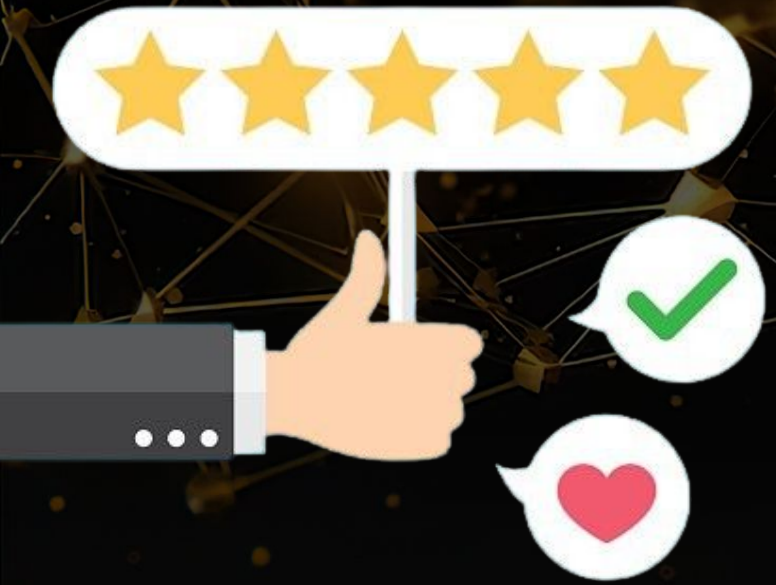
¿Cuánto valor te ha agregado esta sesión?



Absolutamente nada	Poco	Suficiente	Bastante	Más de lo que esperaba

LIDR

ROTI



¡GRACIAS!

LIDR.co, AI-Powered Careers

@lidr_ai

#followtheLIDR